CHAPTER 5 · IESTI05 EDGE AI ENGINEERING

Agentic AI at the Edge

Giving a small model tools

> Qwen3.5 · llama.cpp · no external hardware at all

Marcelo Rovai · UNIFEI

AIEng4D

THE ARC SO FAR

Where this chapter sits

2
Generative AI
llama.cpp from source, llama-server, the openai client

›

3
Multimodal
The same model, given eyes: vision projector + camera

›

4
Real World
Dual-brain classifier: sensors, Bridge RPC, Flask

›

5
Agentic
Native tool-calling — the model chooses

The model reasoned in every chapter. It never decided what to do.

Chapter 4's classify() always reads three sensors, always calls the model once, always with the same prompt. That is a pipeline. Today we build a loop.

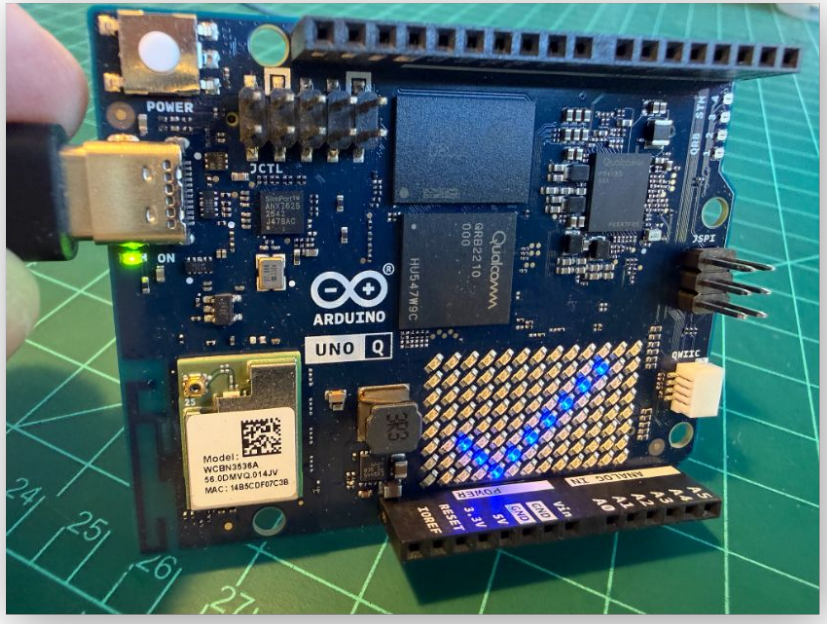
What you are building toward

```
> Check the free space on /home. If there's more than
>  1GB free, show a checkmark on the matrix,
>  otherwise show an X.

[turn 0] tool call: get_system_info({})
[turn 1] tool call: set_led_matrix({'pattern': 'check'})

There's 14 GB free, well over 1 GB, so I've shown a
checkmark on the matrix.
```

No if statement in your code chose that checkmark. Move the threshold from 1 GB to 5 GB and the board draws an X instead — with no code change.



The matrix, obeying a sentence

THE ROUTE

Seven stages, one agent at the end

1	**Agentic?**	Three specific behaviours a plain chat model does not do — and one loop.
2	**Setup**	Nothing new to install. Three flags decide whether this works at all.
3	**Zero code**	Watch an agent loop run in the browser, approving each call by hand.
4	**Tool design**	Six narrow tools, and two design rules that outlive this chapter.
5	**The loop**	Tool schemas, a dispatch table, ~40 lines of Python.
6	**Running it**	The dependable cases, the exact point it breaks, and what the fix costs.
7	**Performance**	Where the time goes, and the one flag worth 3× speed.

HARDWARE NEEDED

None.

No breadboard, no sensors, no wiring. Every tool uses either the Linux filesystem or the board's own onboard LED and 8×13 matrix. Chapters 3 and 4 are deliberately not required.

STAGE 1 OF 7

What makes this "agentic"?

1

Pinned down, it means three specific behaviours — and one loop.

DEFINITION

Three things a plain chat model does not do

1

Decide whether to act at all

'Capital of Brazil?' → answer from what it knows, no tool. 'How much free disk space?' → it cannot know; that answer lives on your board.

2

Pick the tool and fill in the arguments

You hand it a list of tools and their schemas; it chooses. You never write: if 'disk' in prompt: get_system_info()

3

Look at the result and decide what comes next

Another tool call, or a final answer. The decisions chain — and nothing in your code guarantees the chain ever ends.

Nothing guarantees termination. A confused model can call the same tool five times. You cap the iterations yourself and break out with a message rather than trusting it to stop.

THE DISTINCTION

Pipeline versus loop

CHAPTER 4

classify() · a pipeline

Always reads the same three sensor values. Always calls the model exactly once. Always the same prompt shape.

Deterministic, fast, easy to test. You wrote every branch.

CHAPTER 5

run_agent() · a loop

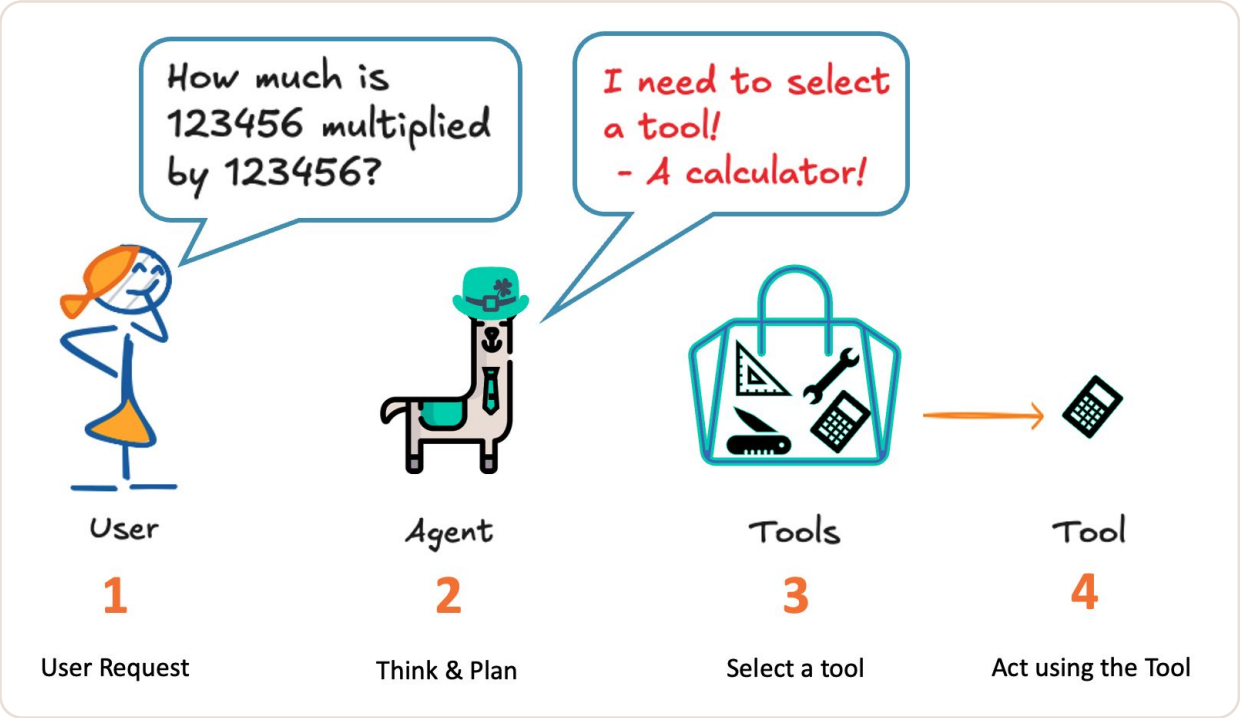
Reads whatever the model asks it to read. Calls the model N+1 times for N tool calls. The conversation grows every turn.

Non-deterministic, slower, harder to test. The model writes the branch.

Neither is better. You do not want a dengue-risk classifier wandering off to check the disk. Know which one your problem needs.

THE MECHANISM

The agent loop



Everything in the dashed box is yours — ordinary Python

The model never runs anything. It emits a function name and a JSON blob of arguments, then stops and waits. Nothing executes unless you execute it.

VOCABULARY

Model · Harness · Agent

MODEL

Qwen3.5

Sits behind llama-server. Reads text, writes text. That is all it does — it cannot open a file, read a sensor or light an LED, and between requests it remembers nothing.

HARNESS

Your ~40 lines

The loop, the dispatch table, the code that formats messages and appends results, the turn limit that stops runaways. Ordinary software. You write all of it.

AGENT

The two together

Neither half is an agent alone. LangChain and the Assistants API are harnesses with more in them — retries, memory, tracing — around exactly this cycle.

Harness bug — a debugger finds it. Model bug — no Python fixes it. Make students name which half is broken before they open an editor.

THE MECHANISM, PRECISELY

Native tools API, not prompt-and-parse.

The old way asked the model to freehand JSON and hoped the shape was right — fragile at 1B and below. The native API moves the structural guarantee into the inference engine: llama.cpp constrains decoding so arguments come back as valid, parseable JSON.

THE ONE MANDATORY FLAG

--jinja, or nothing works.

Tool-call formatting lives inside the chat template, not in a separate code path. Without that flag, tool_calls comes back empty every time and the model describes its intended call in prose instead of making one.

STAGE 2 OF 7

Setup: two models, a few flags

2

Nothing new to install — but three flags decide whether this chapter works at all.

THE EXPERIMENT

Two models, not one

SECTIONS 3–6

Qwen3.5-0.8B · Q8_0

Demonstrates the mechanism — and then hits a wall you can watch it hit.

PARTWAY THROUGH 6

Qwen3.5-2B · Q4_K_XL

The smallest model that reliably uses the mechanism. Roughly 4× the latency.

ONLY ONE AT A TIME

They share port 8081

Know how to stop one before starting the other:

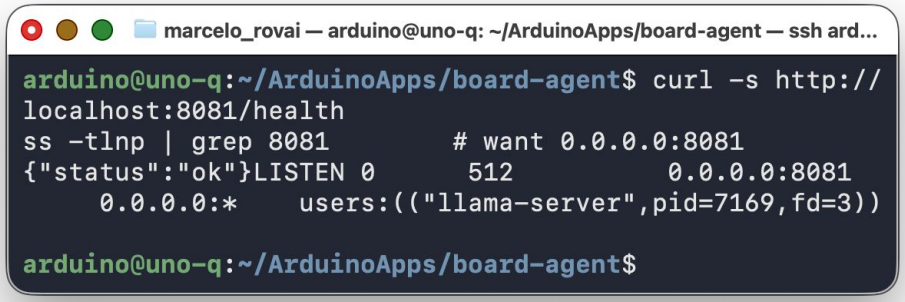
pkill -f llama-server

This is not indecision — the swap IS the experiment. Knowing where a model's capability ends, and what crossing that line costs, is most of the work in edge AI.

CONFIGURATION

The server command

```
$ ~/llama.cpp/build/bin/llama-server \
  --model ~/models/Qwen3.5-0.8B-Q8_0.gguf \
  --host 0.0.0.0 --port 8081 \
  --jinja \
  --reasoning off --reasoning-budget 0 \
  --ctx-size 4096 --threads 4 \
  --batch-size 512 --ubatch-size 64 \
  --predict 1024 \
  --alias qwen3.5-0.8b
```



The server, up

This exact command appears three times in the chapter. If you find yourself editing flags between sections, something has gone wrong.

CONFIGURATION

The flags that matter, and why

1. **--jinja**
   Activates the chat template. Tool-call formatting lives inside it — without this, tool_calls is always empty.

2. **--ubatch-size 64**
   The single biggest performance flag in the chapter — roughly 3× faster answers. Do not judge how slow the agent feels until you set it.

3. **--reasoning-budget 0**
   Thinking and tool-calling interact badly at this size: the model reasons about which tool to call, then emits the reasoning as the answer.

4. **--ctx-size 4096**
   An agent conversation grows with every tool result. Too small and the earliest messages silently truncate — which reads as forgetting.

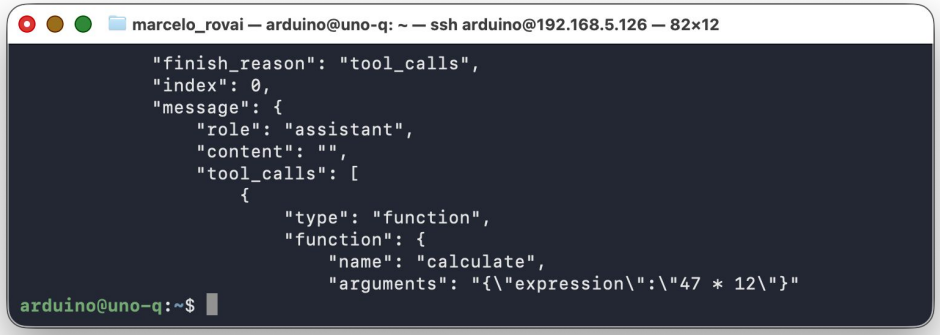
> --alias must match MODEL in main.py. Keep it identical after the 2B swap and no Python changes.

CHECK BEFORE YOU BUILD

Verify tool-calling actually works

```
$ curl -s http://localhost:8081/v1/chat/completions \
  -H 'Content-Type: application/json' -d '{
   "model": "qwen3.5-0.8b",
   "messages": [{"role":"user",
                 "content":"What is 47 times 12?"}],
   "tools": [{"type":"function","function":{
     "name":"calculate", ...}}]
  }' | python3 -m json.tool | grep -A5 tool_calls
```

Do this before writing a line of Python. Plain text back instead of a tool_calls block means everything downstream will fail.



A tool_calls block — correctly configured

STAGE 3 OF 7

First contact: zero code

3

Watch an agent loop run in the llama.cpp WebUI — approving every tool call by hand.

BUILT-IN TOOLS

The --tools allow-list

Built-in tool	What it does	In a workshop?
`read_file`	Reads any file the arduino user can read	Yes
`file_glob_search`	Finds files by name pattern	Yes
`grep_search`	Searches inside file contents	Yes
`write_file`	Creates or overwrites a file	Later, deliberately
`edit_file, apply_diff`	Modifies existing files in place	No
`exec_shell_command`	Runs arbitrary shell commands	Not in a workshop

> The default is: no tools. Nothing is exposed unless you name it on the command line — the others are not merely switched off; as far as the browser is concerned they do not exist.

WHERE CONTROL ACTUALLY LIVES

The panel is a view. The command line is the boundary.

THE COMMAND LINE

--tools decides what exists

Enforced by the server process. Applies to every client, including scripts hitting the API directly. Changing it requires restarting the server.

THE BROWSER PANEL

A view onto that list

Plus per-browser convenience toggles saved in localStorage. Clear your browser data and the checkboxes reset; the server's allow-list does not move.

The most common stumble here is an empty Tools panel — and it is a productive one. A student who only sees checkboxes will believe the browser grants permissions. It does not.

DEMO

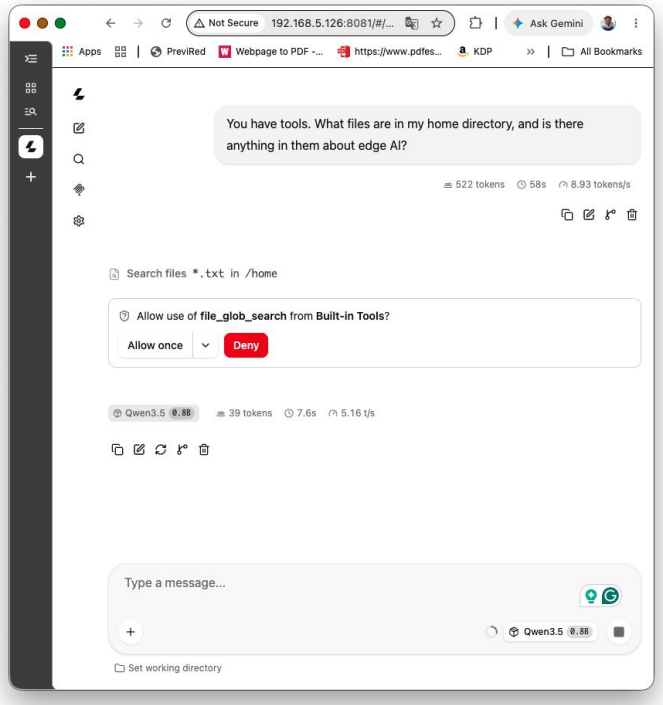
Ask it something it cannot know

```
> "What files are in my home directory, and is there
>  anything in them about edge AI?"

# a confirmation card appears BEFORE anything runs,
# naming the tool and its exact arguments

# expect two calls: one to list, one to search
```

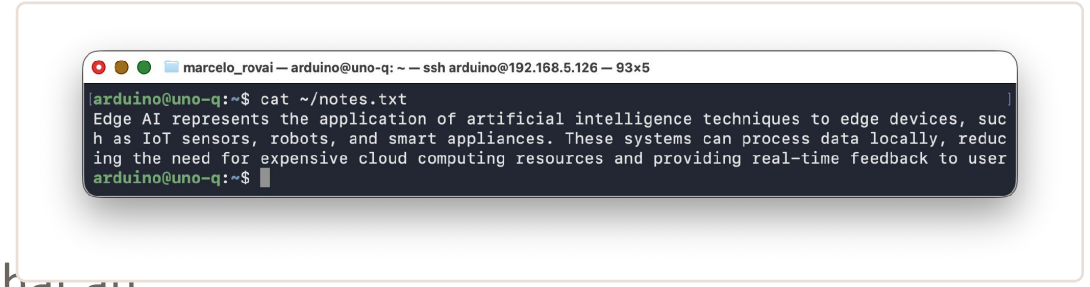
Point at the screen while it runs: send, decide, call, feed the result back, decide again. That is Section 1's diagram happening live.



Approve before anything executes

THE FRICTION IS THE FEATURE

Granting a writing tool meant restarting the server.



You could not have done it from the browser. Expanding what an agent can reach is a deliberate act, with a record of it in your shell history — not a checkbox someone clicks while distracted.

SAFETY

What each layer is actually worth

1

These tools have no directory restriction

read_file will read anything the Arduino user can read. No allow-list of paths. The sharpest contrast with the tools you build next.

2

exec_shell_command runs with your full permissions

Nothing stands between a confused model and a destructive command except the confirmation card. Not in a first workshop.

3

--host 0.0.0.0 is fine only on a network you control

Anyone who can reach the API can drive the same tools — and the confirmation card exists only in the WebUI.

4

'Always allow' is a second line, not the boundary

It lives in browser localStorage. It protects you at the keyboard, not the server.

Read this before your next session with tools enabled. A script hitting /v1/chat/completions directly gets no confirmation prompt at all.

STAGE 4 OF 7

Designing the tool set

4

Six narrow, purpose-built tools — the opposite end of the spectrum from Section 3.

THE TOOL SET

Six tools, each small enough to read in one pass

Tool	What it does	Runs on	Model-controlled input
`get_system_info()`	OS, CPU, RAM, disk	MPU (Python)	None
`list_files(path)`	List a directory	MPU (Python)	A path string
`read_file(path)`	Read a text file	MPU (Python)	A path string
`calculate(expression)`	Arithmetic, via ast	MPU (Python)	An expression string
`set_builtin_led(state)`	Onboard LED on/off	MCU (Bridge)	A boolean
`set_led_matrix(pattern)`	Draw on the 8×13 matrix	MCU (Bridge)	One of 14 fixed values

Look at the last column. It decides how much defensive code each tool needs — and it cuts across intuition.

DESIGN RULE 1

Risk scales with the portion the model controls

LOOKS BROAD. IS NOT THE RISKY ONE.

get_system_info()

It reads system files and reports on the whole machine. But the model supplies zero arguments, so there is nothing to steer. What it reads is fixed by your code, every time.

No model-supplied arguments ⟶ no boundary needed.

LOOKS HARMLESS. NEEDS THE BOUNDARY.

read_file(path) · calculate(expr)

The path comes from the model, and the model's output is downstream of whatever the user typed. An arithmetic string sounds like the safest input imaginable — until you remember the model chose it.

One model-supplied string ⟶ needs a boundary.

Not with how powerful the tool sounds. This rule applies to every agent you will ever build, including MCP servers.

DESIGN RULE 2

A tool's output is read by a model, not a person

FRIENDLY TO A HUMAN

"1.4G free of 9.8G"

The model has to extract a number and reason about units before it can answer 'is there more than 5 GB free?' — a step small models routinely get wrong.

WHAT THE TOOL ACTUALLY RETURNS

"disk_free_gb": 1.4

One value, one unit, already numeric. The comparison the model has to make is now the only thing it has to do.

Format for the reader in your final answer, never in the tool result. There is a latency argument too: verbose results cost reading time on every subsequent turn — which is why read_file caps at 4 KB.

CONSEQUENCES IN CODE

Two boundaries, drawn on purpose

```
# 1 — file tools resolve inside a fixed workspace
def _resolve_in_workspace(path):
    p = (WORKSPACE_ROOT / path).resolve()
    if not p.is_relative_to(WORKSPACE_ROOT):
        raise ValueError("outside workspace")
    return p

# 2 — calculate walks a restricted AST; no eval()
# Allowed: numbers, + - * / **, parens, unary minus.
# Anything outside the whitelist raises.
ALLOWED = (ast.Expression, ast.Constant,
           ast.BinOp, ast.UnaryOp, ...)
```

The workspace boundary protects nothing FROM you — you own the board and have a shell. It is there because the MODEL is choosing that path now.

THE CHEAPEST RELIABILITY WIN

Put the valid inputs in the schema.

llama.cpp constrains sampling to the grammar the schema implies. The model cannot produce an invalid pattern — not 'will be rejected if it does', but cannot emit one in the first place. Validating in Python catches a bad value after the tokens were already spent.

STAGE 5 OF 7

Building the agent loop

5

Tool schemas, a dispatch table, forty lines of Python, and a Flask page to talk to it.

MAIN.PY

The loop itself

```
for turn in range(MAX_TURNS):
    response = client.chat.completions.create(
        model=MODEL, messages=messages, tools=TOOLS,
        temperature=0.3, max_tokens=512)
    msg = response.choices[0].message

    if not msg.tool_calls:
        return {"answer": msg.content}      # done

    messages.append({"role": "assistant",
        "content": msg.content or "",   # None breaks replay
        "tool_calls": [tc.model_dump()
                       for tc in msg.tool_calls]})

    for tc in msg.tool_calls:
        args = json.loads(tc.function.arguments or "{}")
        result = DISPATCH[tc.function.name](**args)
        messages.append({"role": "tool",
            "tool_call_id": tc.id, "content": result})
```

Note the or "" — on a pure tool-call turn content is None, and some OpenAI-compatible servers reject a null content field when it is replayed back. That is a 30-minute bug.

MAX_TURNS = 6

A hard cap that reports the runaway.

Nothing in your code guarantees the model stops. The cap does not hide the problem — it surfaces it. You will watch it catch two real runaways shortly.

WHAT RUNS WHERE

Three details that explain most of what goes wrong

1

llama-server is on the host; the agent is in a container

The agent reaches the model at a gateway address. A loopback-bound server is invisible from inside — hence --host 0.0.0.0.

2

Two ports, two different things

8081 is the model. 7000 is your agent. Confusing them is the most common wrong turn here — same as Chapter 4.

3

Only two tools cross to the MCU

The other four resolve entirely in Python — which is why they keep working when the Bridge does not.

> Make students say the port numbers out loud. Really.

OPERATING IT

Three terminals, one browser

TERMINAL 1

The model

llama-server, in the foreground so you can watch it load and see requests arrive.

TERMINAL 2

The agent's thinking

arduino-app-cli app logs . --follow

The [turn N] tool call lines scroll here. This is the one to keep on the projector.

TERMINAL 3

Free — where you type

python3 ask.py "what is 234 × 17, minus 9?"

Or use the web page from any device on the network.

http://<board-ip>:7000/ gives a text box, and each answer appears with the tool calls that produced it. For a workshop that is the better surface — students drive it from their own laptops.

STAGE 6 OF 7

Running it — and watching it fail

6

The dependable cases at 0.8B, the exact point where it breaks, and what one changed file path buys you.

0.8B, THE DEPENDABLE CASES

One question, one tool, one answer — about 20 seconds

TOOLS THAT FIRE

Three reliable demos

get_system_info — 'free disk space?'

calculate — '234 × 17, minus 9?'

set_builtin_led — 'turn on the LED'

THE NEGATIVE RESULT

'Capital of Brazil?'

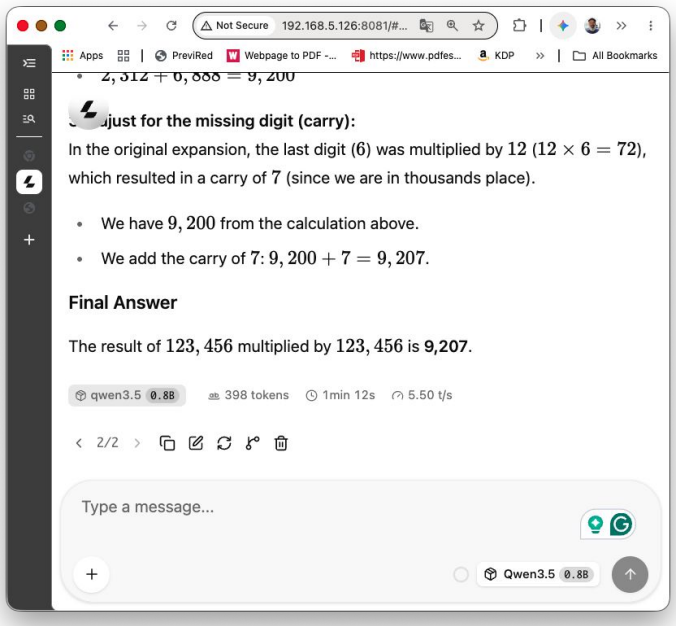
Returns Brasília straight from what the model knows. No tool call, no round trip, no latency. Watch terminal 2: it stays empty.

Deciding NOT to call a tool is part of the job.

Demonstrate the negative case explicitly — a student who has only seen tools fire will assume an agent always uses one.

THE ARGUMENT FOR TOOLS, RUN LIVE

123456 × 123456, with and without a calculator



Straight to the model: over a minute of reasoning, and a wrong answer

Through the agent: the model calls calculate() and reads back the exact product — correct, in a couple of seconds. Same board, same model, same question.

THE CAPABILITY BOUNDARY

Now push slightly harder, and watch it break

FAILURE 1

It did not know it was finished

'List the workspace.' The correct answer arrived at turn 0.

Then it kept going: read both files, checked system info, toggled the LED, drew on the matrix, did unrelated arithmetic, and hit MAX_TURNS without ever answering.

FAILURE 2

It could not reason over a value

1.4 GB free is declared 'less than 1 GB' and it draws an X.

Another run computed 1.4 / 5 = 0.28, decided that meant something, and drew the digit 0 four times running.

Neither failure is in your code. Every tool returned correct data, the dispatch table worked, the Bridge worked, and MAX_TURNS caught both loops cleanly. What failed is the model's judgement.

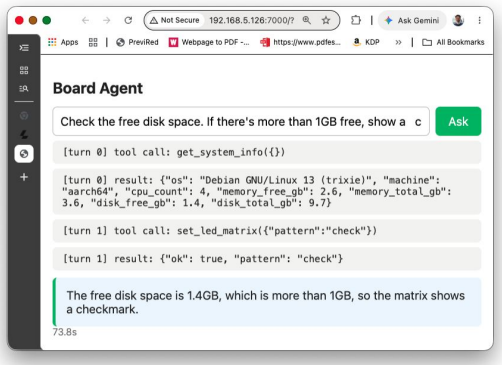
THE FIX, PRICED

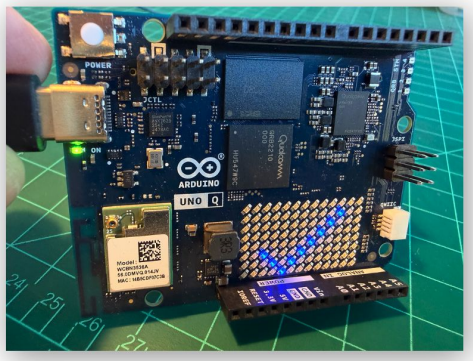
Crossing the boundary: change one file path

```
--model ~/models/Qwen3.5-2B-UD-Q4_K_XL.gguf \
  ...
  --alias qwen3.5-0.8b      # deliberately unchanged

# same flags, same alias,
# not one line of Python or C++ changes
```

The first request after loading takes about 200 seconds while the model works through the prompt cold. Do not judge it on that one.





Threshold 1 GB ⟶ checkmark

STAGE 7 OF 7

Performance and reliability

7

Where the time goes, the one flag that changes everything, and what capability costs.

LATENCY

One flag makes answers three times faster.

--ubatch-size 64. Between requests llama-server keeps a KV cache of the prompt prefix, but can only reuse up to a micro-batch boundary — so every request recomputes as much as ubatch-size tokens, however well the prefix matched. Pure waste, paid on every question.

LATENCY

Where the time actually goes

1

An agent answer is N + 1 inference calls

One call to decide on each tool, one more to write the answer. A single-tool question costs two; a three-tool chain costs four.

2

Reading dominates, not writing

prompt eval 7107 ms / 68 tok · eval 2664 ms / 15 tok. A typical answer generates 15–30 tokens and reads several hundred.

3

It matters MORE at 2B

The same wasted tokens cost 2.4× as much each. Left at the default there, a two-call answer runs past five minutes.

Measure it yourself: ask a question twice at 512, note the second time; restart at 64 and repeat. The most useful ten minutes in this chapter.

NEGATIVE RESULTS

What did not help — and why recording it matters

1

--cache-reuse N

Exactly the right idea. This build logs 'cache_reuse is not supported by this context, it will be disabled'. Check your own startup log.

2

-np 1 (a single slot)

No measurable change. Slot contention was not the problem.

3

--threads 3, leaving a core for Flask

Clearly worse: 7.4 vs 9.4 tok/s, about 27% slower. Keep all four.

4

Trimming the system prompt

Cutting ~115 tokens shortened the FIRST request and changed nothing afterward — the reprocessed portion is fixed at 68 tokens either way.

Each of these sounds plausible. Measuring them is the difference between engineering and folklore. Concise tool descriptions are a reliability knob, not a latency one.

THE TRADE

What model size costs — measured on the same board

	Qwen3.5-0.8B (Q8_0)	Qwen3.5-2B (Q4_K_XL)
Prompt eval	10.6 tok/s (95 ms/tok)	4.2 tok/s (241 ms/tok)
Generation	5.6 tok/s (178 ms/tok)	2.5 tok/s (398 ms/tok)
First request after load	~66 s	~199 s
Single-tool answer	~20 s	~40 s
Threshold demo (3 calls)	fails	80 s
Multi-step reasoning	unreliable	works

Roughly 2× slower, and it works. Ship 0.8B when one tool and 20 seconds is enough — most of what an embedded agent does. Ship 2B when the task involves the word 'if'.

GENERALISING THE FAILURES

Why small models fail at this specifically

FAILURE 1

Knowing when it is done

After each tool result the model sees the full tool list again and must decide no further call is needed. That is harder than choosing a tool in the first place — and small models resolve it by calling something.

FAILURE 2

Reasoning over a value

'Is 1.4 more than 5' is trivial. Extracting 1.4 from a sentence, deciding its unit, and THEN comparing is not. Returning numbers removes the first two steps — and 0.8B still failed.

Good tool design raises the ceiling. It does not replace model capability. The native tools API constrains the shape of a call once the model decides to make one — it does not make the model decide correctly, or decide to stop.

WHEN IT BREAKS

The failures you will actually hit

1

tool_calls is always empty

The server was started without --jinja. Tool-call formatting lives in the chat template.

2

ModuleNotFoundError: openai

You ran python3 python/main.py from an SSH shell. Use arduino-app-cli app start . — the deps live in the environment it manages.

3

The tool result is right but the answer is wrong

A model problem, not a harness problem. If the result JSON is correct, no Python will fix it.

4

The LED tools fail while the others work

The Bridge, not the agent. Four tools resolving fine while two hang is the signature.

Also: /healthz returning connection refused (server down, or bound to 127.0.0.1), infinite loops (MAX_TURNS working, not crashing), and edits doing nothing (app restart needed).

CARRY THESE OUT OF CHAPTER 5

Five things

1. **An agent is a model plus a harness you write**
   Knowing which half a bug lives in is most of debugging one.
2. **A tool's risk is the portion the model controls**
   Powerful is not the same as risky.
3. **Tool output is read by a model, not a person**
   Return disk_free_gb: 1.4, not '1.4G free of 9.8G'.
4. **Express valid inputs in the schema, not in Python**
   An enum cannot be violated — the invalid tokens were never possible.
5. **Capability is bought with latency, always**
   0.8B demonstrates the mechanism; 2B is the smallest that reliably uses it, at ~4× the time.

Building a small harness by hand is the fastest way to understand what LangChain, the Assistants API and every other agent framework are actually doing.

RESOURCES

Where to read further

Resource	Where
`llama.cpp — function-calling docs`	github.com/ggml-org/llama.cpp/blob/master/docs/function-calling.md
`llama-server flags, --tools and CORS`	github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md
`Qwen3.5 function-calling guide (Unsloth)`	unsloth.ai/docs/models/qwen3.5
`Edge ML Made Easy — Building Agents with SLMs`	mjrovai.github.io/EdgeML_Made_Ease_ebook/
`QClaw — an agent that writes, compiles and flashes`	github.com/laurenvil/Uno-QClaw
`Arduino_LED_Matrix library`	github.com/arduino-libraries/Arduino_LED_Matrix

Ch.4's sensor and actuator functions drop into this loop unchanged — add three entries to TOOLS and three lines to DISPATCH, and the MODEL decides when to turn the red LED on.

Questions?

Prof. Marcelo J. Rovai

rovai@unifei.edu.br

UNIFEI — Federal University of Itajubá, Brazil

AiEng4D — Academic Network Co-Chair

EdgeAIP — Academia-Industry Partnership Co-Chair